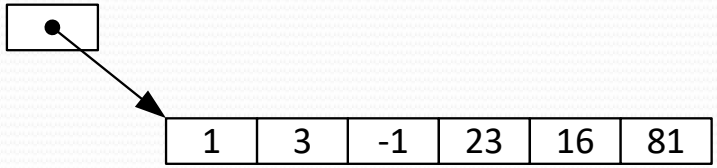
Глава 4. Массивы, строки и структуры языка C#

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Дисциплина Алгоритмизация и программирование
Лектор: д.т.н., проф. Иванова Галина Сергеевна

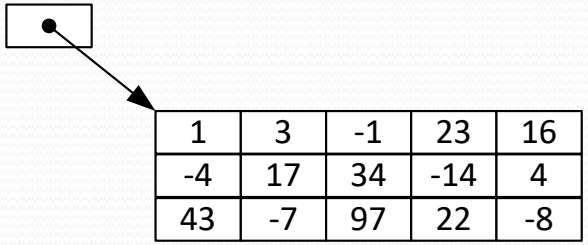
4.1 Массивы. Типы массивов и элементов

Различают массивы:

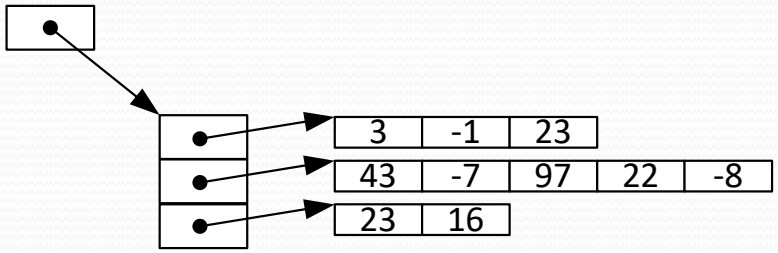
- одномерные;



- прямоугольные;



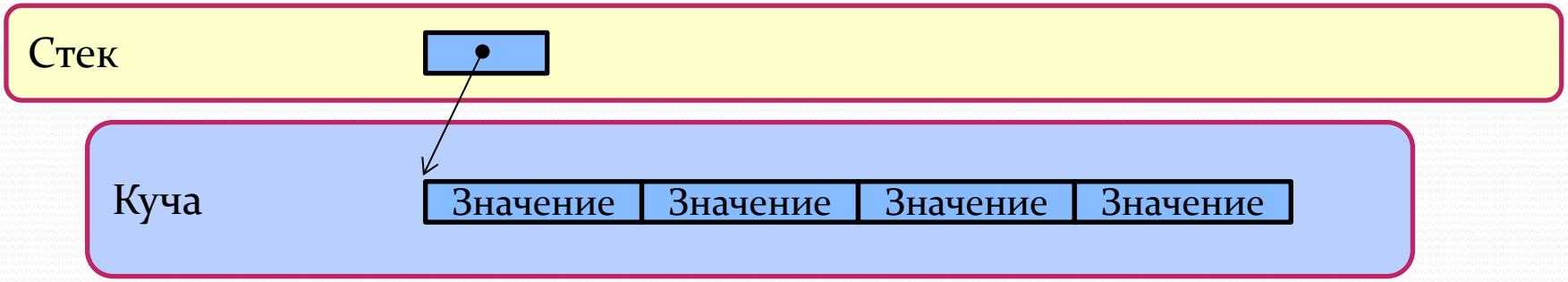
- ступенчатые.



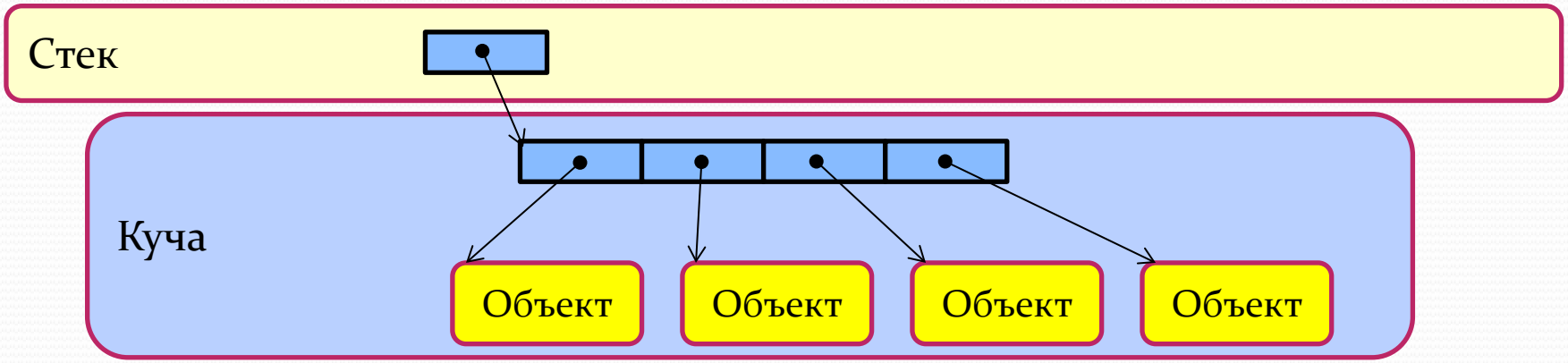
Массивы с элементами-значениями и элементами-объектами

Массивы могут включать элементы типа-значения и типа-ссылки.

Массив данных типов- значений



Массив данных ссылочного типа



4.1.1 Одномерные массивы

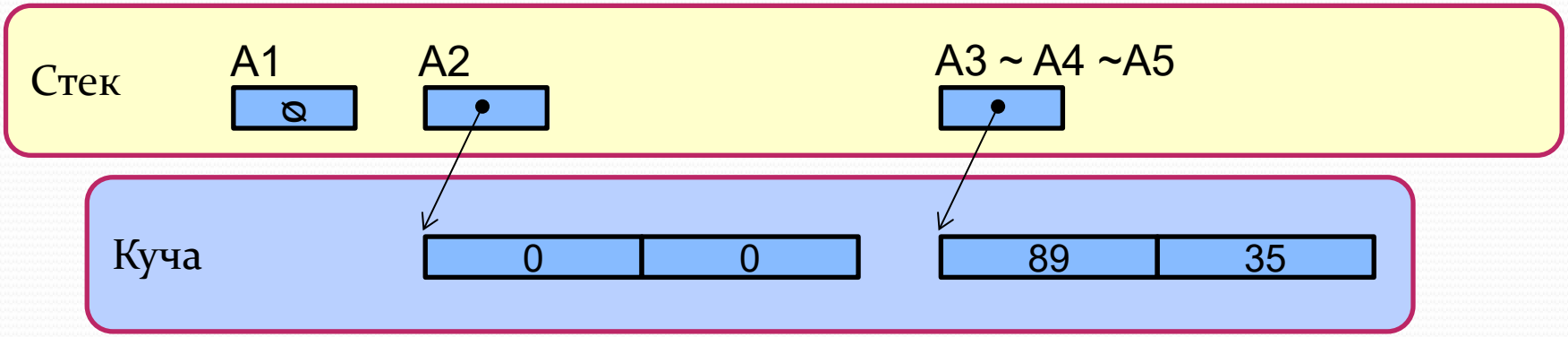
Синтаксис объявления одномерного массива:

Тип_элемента [ ] Имя [=

[new Тип_элемента[[Размерность]]]{Список_инициализаторов}];

Примеры массивов типов-значений:

```
int[] A1;                          // без выделения памяти под элементы
int[] A2 = new int[2];             // с инициализацией элементов нулями
int[] A3 = {89,35};                // с инициализацией элементов
int[] A4 = new int[]{89,35};
int[] A5 = new int[2]{89,35};
```



Одномерные массивы ссылочных типов

Синтаксис объявления одномерного массива:

Тип_элемента [ ] Имя [=

[new Тип_элемента[[Размерность]]]{Список_инициализаторов}];

При объявлении массивов объектов *следует отдельно создавать объекты*.

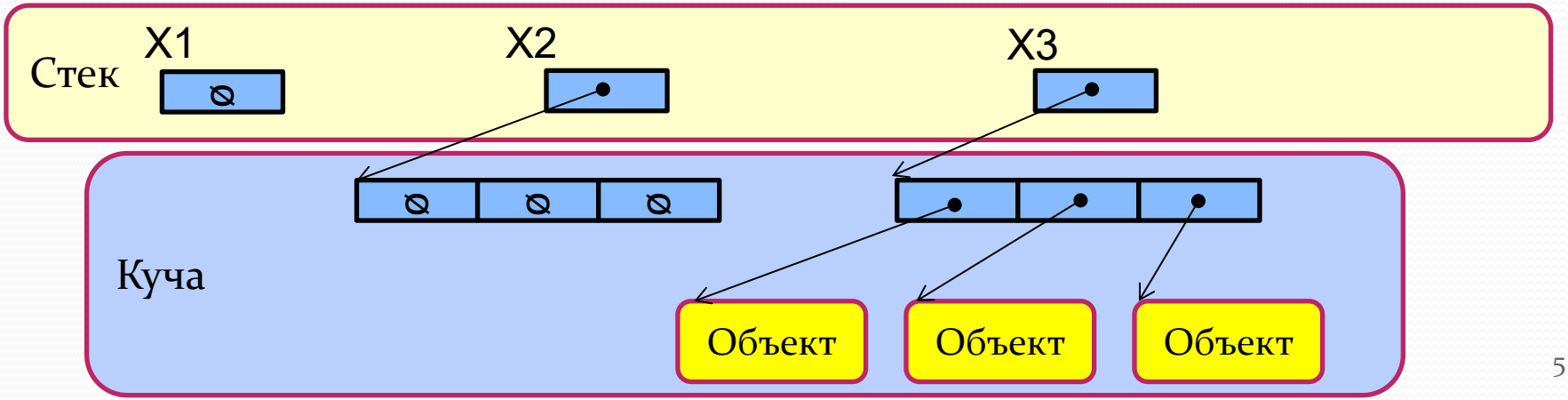
Примеры:

а) `A[] X1;` // создан указатель на массив

б) `A[] X2 = new A[3];` // создан массив из 3 указателей

```
for (int i=0;i<3;i++)
    X2[i] = new A(int.Parse(Console.ReadLine()));
```

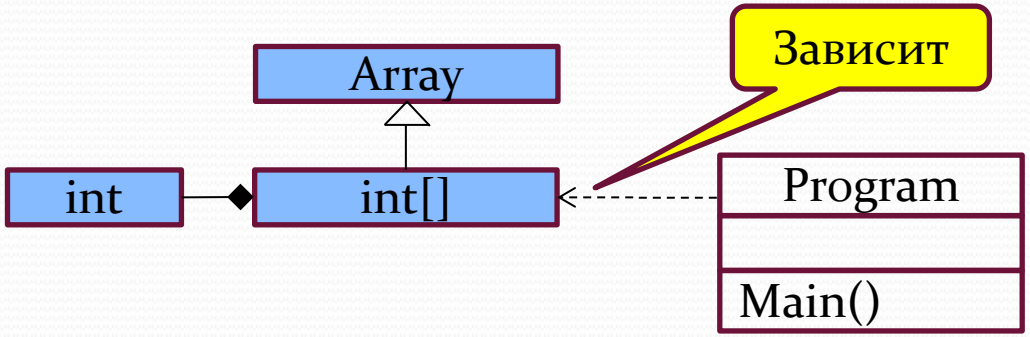
в) `A[] X3 = {new A(1),new A(2),new A(3)};`

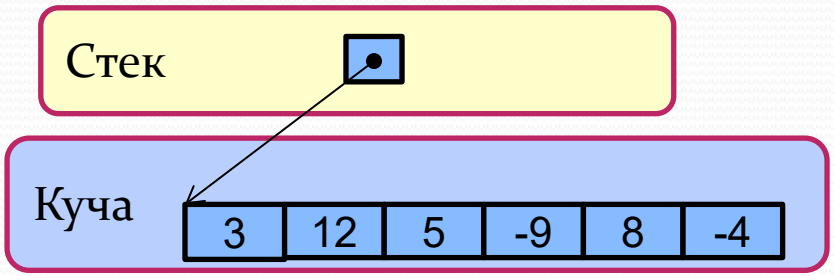


Пример

Ex4.01

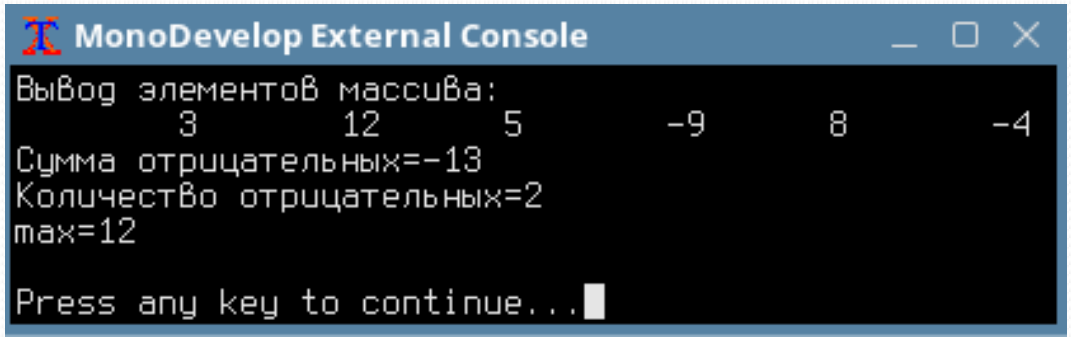
```
using System;
namespace Ex4_01
{
    class Program
    {
        static void Main()
        {
            const int n=6;
            int[] a = new int[n]{ 3, 12, 5, -9, 8, -4};
            Console.WriteLine("Вывод элементов массива:");
            for(int i = 0; i < n; ++i)
                Console.Write("\t" + a[i]);
            Console.WriteLine();
           int num = 0;
```





Пример (продолжение)

```
long sum = 0;
for(int i=0;i<n;++i)
    if(a[i]<0)
    {
        sum += a[i]; ++num;
    }
Console.WriteLine("Сумма отрицательных="+sum);
Console.WriteLine("Количество отрицательных="+num);
int max = a[0];
for(int i = 1;i < n;++i)
    if ( a[i] > max ) max = a[i];
        Console.WriteLine( "Max=" + max);
    }
  }
}
```



4.1.2 Оператор foreach

Оператор foreach осуществляет последовательное чтение элементов коллекции (массива, строки и т.п.).

Синтаксис оператора:

`foreach` (Тип Имя `in` Выражение) Тело_цикла

Примеры:

1) Вывод элементов одномерного массива:

```
foreach (int x in a) Console.WriteLine(x);
```

2) Вывод элементов прямоугольного и ступенчатого массивов:

```
foreach (int[] x in a)
{
    foreach (int y in x) Console.Write("{0,6}", y);
    Console.WriteLine();
}
```

Пример

Ex4.04

```
MonoDevelop External Cons...
Введите количество элементов(до 15)5
  13   1   3  -4 -13
 -13  -4   1   3  13
  13   3   1  -4 -13

Press any key to continue...
```

```
static void Main()
{
    Console.Write("Введите количество элементов(до 15)");
    int n = int.Parse(Console.ReadLine());
    Random r = new Random();  // создание экземпляра класса Random
    int[] a = new int[n];     // объявление массива размерности n
    for (int i = 0; i < n; i++)
    {  a[i] = r.Next(-20, 20); // инициализация случайными цел. числами
       Console.Write("{0,4}", a[i]); // форматный вывод
    }
    Console.WriteLine();
    Array.Sort(a);            // упорядочивание по возрастанию
    foreach (int x in a) Console.Write("{0,4}", x);
    Console.WriteLine();
    Array.Reverse(a);         // реверс массива
    foreach (int x in a) Console.Write("{0,4}", x);
    Console.WriteLine();
}
```

4.1.3 Прямоугольные массивы

Прямоугольные массивы могут иметь различную размерность.

Синтаксис объявления двумерного массива:

Тип_элемента[,] Имя [=

[new Тип_элемента] [[Размерность1], [Размерность2]]

{{Список_инициализаторов},

{{Список_инициализаторов}}

}];

Примеры:

```
int[,] B1;                          // без выделения памяти под элементы
int[,] B2 = new int[2,3];           // с инициализацией элементов нулями
int[,] B3 = {{4,5,6},{89,35,1}};    // с инициализацией элементов
int[,] B4 = new int[,] {{4,5,6},{89,35,1}};
int[,] B5 = new int[2,3] {{4,5,6},{89,35,1}};
```

Пример

Ex4.02

```
using System;
namespace Ex4_02
{
   class Class1
   {
      static void Main()
      {
         int m,n,i,j;
         n = Convert.ToInt32(Console.ReadLine());
         m = Convert.ToInt32(Console.ReadLine());
         int[,] a = new int[n,m];
         for (i=0;i<n;i++)          //foreach(int[] i in a)
            for (j=0;j<m;j++)       //    foreach(int j in i)
               a[i,j]=              //        j = …
                     Convert.ToInt32(Console.ReadLine());
```

Пример (продолжение)

```
        int[] s = new int[n];
        for (i=0;i<n;i++)
        {
            for(j=0;j<m;j++)
            {
                s[i]+=a[i,j];
                Console.Write("\t{0}",a[i,j]);
            }
            Console.WriteLine("\tsum = {0}",s[i]);
        }
    }
  }
}
```

foreach использовать неудобно из-за связной индексации a и s!

```
2
3
1
2
3
4
5
6
        1       2       3       sum = 6
        4       5       6       sum = 15

Press any key to continue...
```

4.1.4 Ступенчатые массивы

Синтаксис объявления ступенчатого массива:

Тип_элемента[][] Имя = new Тип_элемента [Размерность1][];

Имя[0] = new int[Размерность];

{Имя[1] = new int[Размерность];}

или

Тип_элемента[][] Имя = {new Тип_элемента[Размерность][]

{, new Тип_элемента[Размерность][]}};

Примеры:

```
1) int[][] a = new int[3][];
       a[0] = new[5];
       a[1] = new[3];
       a[2] = new[2];
2) int[][] a = {new int[5],new int[3],int[2]};
```

Обращение к элементам **a[i][j]**.

Различие между прямоугольными и ступенчатыми массивами

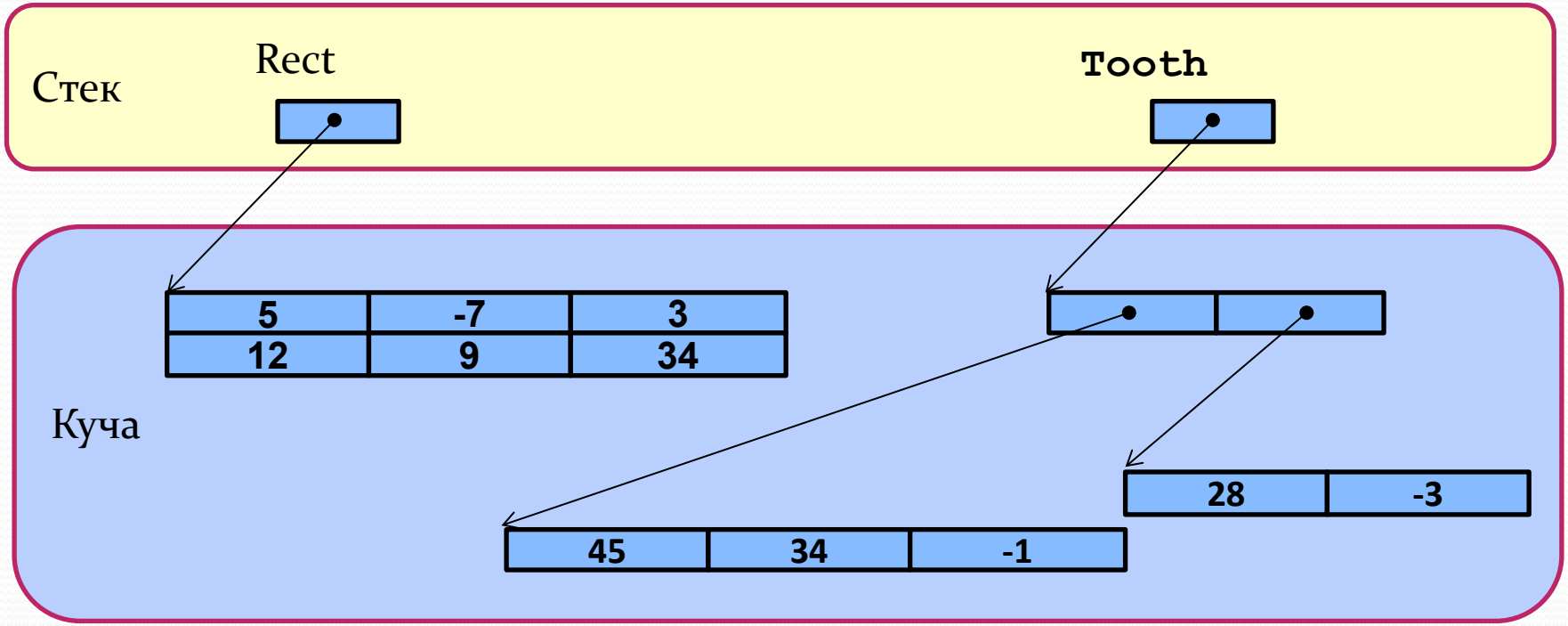
Различие в синтаксисе связано с различным размещением в памяти.

Прямоугольный массив:

```
int[,] Rect = new int[2,3]
  {{5 -7 3},{12 9 34}};
```

Зубчатый массив:

```
int[][] Tooth = new int[2][];
 Tooth[0] = new int[3]{45,34,-1};
 Tooth[1] = new int[2]{28,-3};
```



4.1.4 Класс – родитель массивов System.Array

Реализует интерфейсы: IList, ICloneable и т.п. Имеет фиксированный размер.

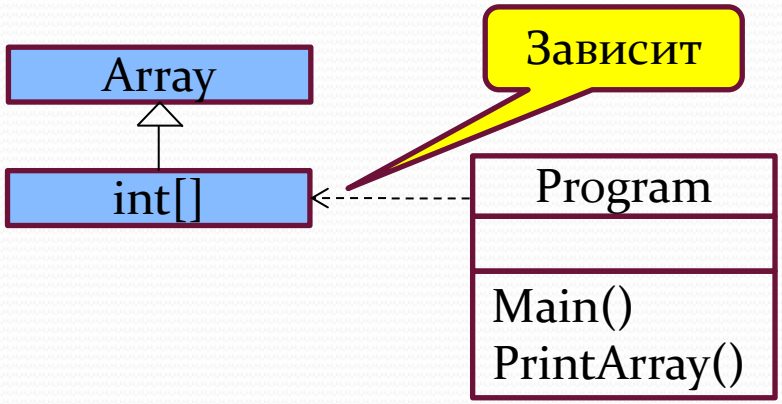
Компонент	Вид	Описание
Length	Свойство	Количество элементов массива (значение фиксировано)
Rank	Свойство	Количество размерностей массива
BinarySearch	Статический метод	Двоичный поиск в отсортированной массиве
Clear	Статический метод	Присваивание элементам массива значений по умолчанию (нулевых)
Copy	Статический метод	Копирование заданного диапазона значений одного массива в другой
CopyTo	Метод	Копирование одномерного массива в другой
GetValue	Метод	Получение элемента массива
SetValue	Метод	Запись элемента массива
IndexOf	Статический метод	Поиск первого вхождения элемента в массив
LastIndexOf	Статический метод	Поиск последнего вхождения элемента в массив
Sort	Статический метод	Упорядочение элементов одномерного массива
Reverse	Статический метод	Изменение порядка элементов массива на обратный
Resize	Статический метод	Меняет размер одномерного массива

Метод System.Array.Sort

Метод	Описание
System.Array.Sort(arrayName)	Сортировка массива по возрастанию
System.Array.Sort(arrayName, startIndex, Length)	Сортировка части массива, где startIndex – индекс начального элемента, Length – количество элементов
System.Array.Sort(array1, array2)	Сортировка массива 2 по ключам в массиве 1
System.Array.Sort(Array, IComparer)	Сортировка массива объектов, для сравнения объектов используется метод IComparer
System.Array.Sort(Array, startIndex, Length, IComparer)	Сортировка части массива объектов

Пример

Ex4.03

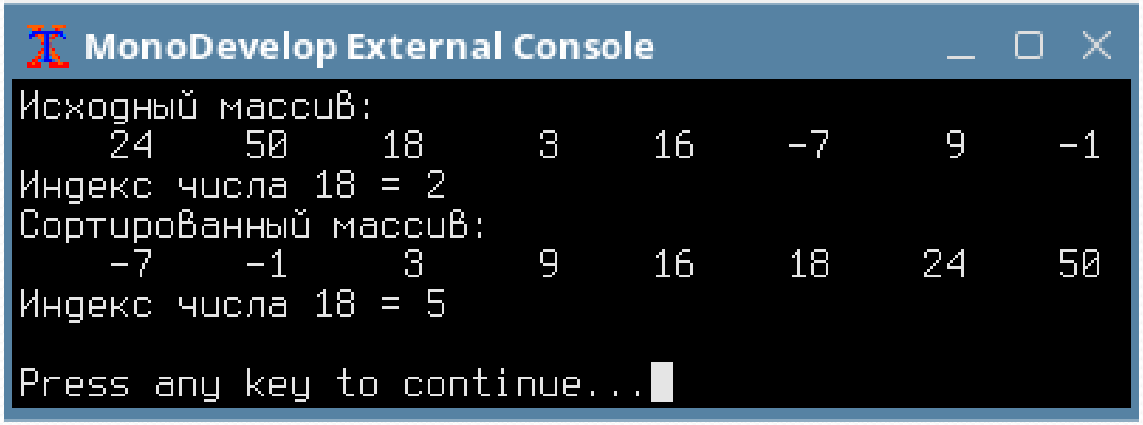



```
using System;
namespace Ex4_03
{
    class Program
    {
        static void PrintArray(string header,Array a)
        {
            Console.WriteLine(header);
            for(int i=0;i<a.Length;++i)
            Console.Write("{0,6}", a.GetValue(i));
            Console.WriteLine();
        }
```

Обеспечивает вызов без объекта из статического метода

Пример (окончание)

```
    static void Main()
    {
        int[] a = {24, 50, 18, 3, 16, -7, 9, -1};
        PrintArray("Исходный массив:", a);
        Console.WriteLine("Индекс числа 18 = "+
                                    Array.IndexOf(a,18));
        Array.Sort(a);
        PrintArray("Сортированный массив:", a);
        Console.WriteLine("Индекс числа 18 = "+
                                    Array.IndexOf(a,18));
    }
  }
}
```



4.2 Символы и строки

Кодировка UTF16 (2 байта – 1 символ).

Основные методы класса System.Char

Компонент	Описание
`GetUnicodeCategory`	Возвращает категорию символа(цифры - DecimalDigitNumber , римские цифры - LetterNumber , разделители строк - LineSeparator , буквы в нижнем регистре – LowercaseLetter и т.п.
`IsControl`	Возвращает true, если символ управляющий
`IsDigit`	Возвращает true, если символ – десятичная цифра
`IsLetterOrDigit`	Возвращает true, если символ – буква или цифра
`IsNumber`	Возвращает true, если символ – дес. или шест. число
`IsPunctuation`	Возвращает true, если символ – знак препинания
`IsSeparator`	Возвращает true, если символ – знак разделитель
`IsWhiteSpace`	Возвращает true, если символ – пробел или #13 или #10
`IsLower, IsUpper`	Возвращает true, если символ – в нижнем или верхнем регистре
`Parse`	Преобразует строку размером 1 символ в двухбайтовый символ
`ToLover, ToUpper`	Преобразует символ в нижний или верхний регистр
`MaxValue,MinValue`	Возвращает символы с макс. и мин. кодами

Пример Вывод кодов введенных символов

Program
Main() PrintChars()

```
using System;
namespace Ex04_20 {
  class Program {
     static void PrintChars(string s) {
        Console.WriteLine($"\"{s}\".Length = {s.Length}");
        for (int i = 0; i < s.Length; i++) {
            Console.WriteLine($"s[{i}] = '{s[i]}'
                                    ('\\u{(int)s[i]:x4}')");
        }
        Console.WriteLine();
     }
     public static void Main (string[] args) {
        Console.WriteLine ("Введите строку: ");
        string s = Console.ReadLine();
        Console.WriteLine ("Введена строка: "+s);
        PrintChars (s);
     }
  }
}
```

Пример

MonoDevelop External Cons...

```
Введите строку:
Hello - Привет.
Введена строка: Hello - Привет.
"Hello - Привет.".Length = 15
s[0] = 'H' ('\u0048')
s[1] = 'e' ('\u0065')
s[2] = 'l' ('\u006c')
s[3] = 'l' ('\u006c')
s[4] = 'o' ('\u006f')
s[5] = ' ' ('\u0020')
s[6] = '-' ('\u002d')
s[7] = ' ' ('\u0020')
s[8] = 'П' ('\u041f')
s[9] = 'р' ('\u0440')
s[10] = 'и' ('\u0438')
s[11] = 'в' ('\u0432')
s[12] = 'е' ('\u0435')
s[13] = 'т' ('\u0442')
s[14] = '.' ('\u002e')


Press any key to continue...
```

4.2.1 Строки – объекты класса string

Строка string – неизменяемый тип данных! При каждом изменении создается новая строка.

Примеры объявления строк:

```
string s;                    // ссылка без памяти
string q = null;             // ссылка без памяти
string message3 = System.String.Empty;  // строка пустая ("")
string t = "Это строка"; // строка инициализирована литералом
```

Использование конструкторов строки:

```
string u = new string(' ', 20);  // строка – 20 пробелов
char[] a = {'0', '0', '0'};      // символьный массив
string v = new string(a);        // создание строки из символьного
                                 // массива
```

Операции на строками string

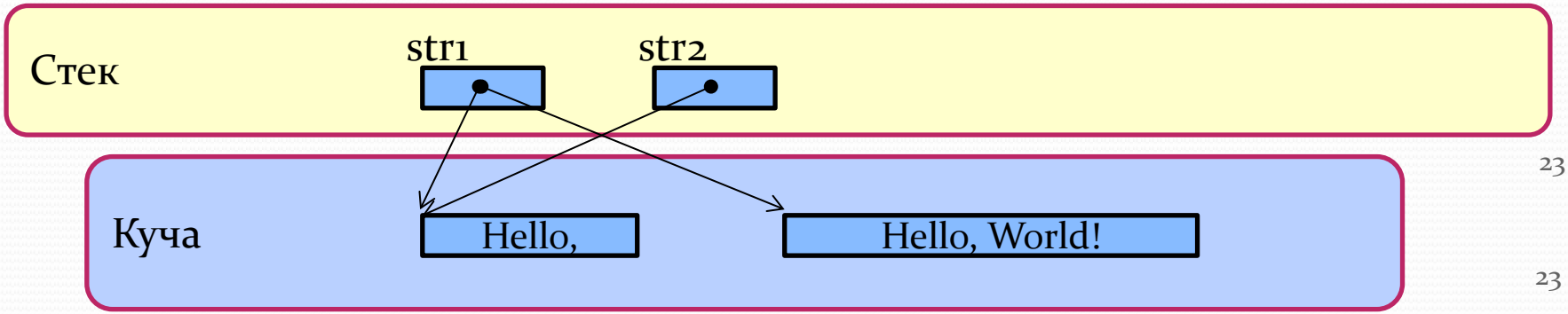
Операции:

- = - присваивание строк(ссылок на строки);
- == - проверка на равенство строк (посимвольное сравнение);
- != - проверка на неравенство;
- [ ] – обращение к символу по индексу (только для чтения символов);
- + - сцепление (конкатенация) строк.

Пример:

```
string str1 = "Hello, ";
string str2 = str1;
str1 += "World!";
System.Console.WriteLine(str2);  // Вывод: Hello,
```

Проблема ссылок и неизменяемых строк



23

Поля и методы класса string

Компонент	Вид	Описание
`Compare`	Статический метод	Сравнение строк (подстрок), с учетом и без учета регистров
`CompareOrdinal`	Статический метод	Сравнение строк (подстрок) по кодам символов
`CompareTo`	Метод	Сравнение текущей строки с другой строкой
`Concat`	Статический метод	Сцепление произвольного количества строк
`Copy`	Статический метод	Создание копии строки
`Empty`	Статическое поле	Пустая строка (только для чтения)
`Format`	Статический метод	Форматирование в соответствие с заданными спецификациями формата
`IndexOf`, `IndexOfAny`, `LastIndexOf`, `LastIndexOfAny`	Методы	Определение первого и последнего вхождения заданной подстроки или любого символа из заданного набора
`Insert`	Метод	Вставка подстроки в строку с указанного индекса
`Intern`, `IsInterned`	Статические методы	Возвращает ссылку на строку, если такая уже существует. Если строки нет, то Interned добавляет строку к пулу, а IsInterned возвращает Null

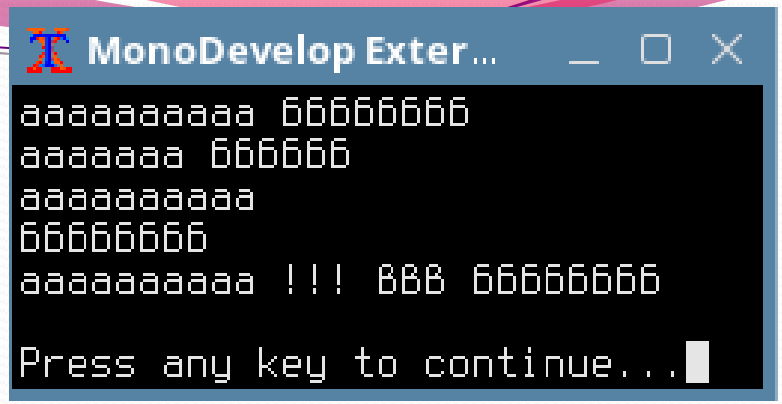
Методы класса string (продолжение)

Компонент	Вид	Описание
Join	Статический метод	Слияние массива строк в единую строку с разделителями строк
Length	Свойство	Длина строки
PadLeft, PadRight	Методы	Выравнивание строки по левому или правому краю путем добавления пробелов
Remove	Метод	Удаление подстроки, начиная с заданной позиции
Replace	Метод	Замена всех вхождений подстроки или символа на другую подстроку или символ
Split	Метод	Разделение строки на подстроки по заданным разделителям и помещения их в массив строк
Substring	Метод	Выделение подстроки с заданной позиции
ToCharArray	Метод	Преобразование строки в массив символов
ToLower,ToUpper	Методы	Преобразование строки к нижнему или верхнему регистру
Trim,TrimStart, TrinEnd	Методы	Удаление пробелов в начале и в конце строки или только в начале или в конце

Пример

Ex4.05

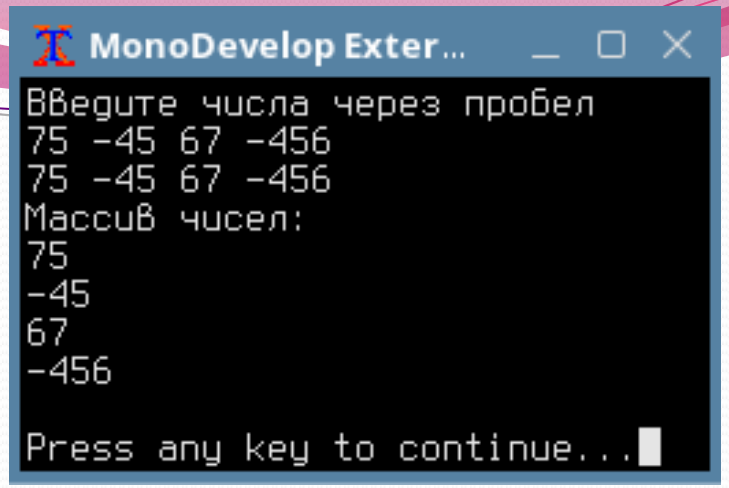
```
class Program
{
    static void Main()
    {
        string s = "aaaaaaaaa ббббббб";
        Console.WriteLine(s); // вывод
        string sub = s.Substring(3).Remove(12, 2) ;
        Console.WriteLine(sub);
        string[] mas = s.Split(' ');
        foreach (string s2 in mas)
            Console.WriteLine(s2);
        string joined = string.Join(" !!! ввв ", mas);
        Console.WriteLine(joined);
    }
}
```



Пример

Ex4.06

```
using System;
namespace Ex4_06
{ class Program
  {  static void Main()
     {
        Console.WriteLine ("Введите числа через пробел");
        string s = Console.ReadLine ();
        string[] ss = s.Split (' ');   // разбивка на строки
        foreach (string b in ss) Console.Write (b + ' ');
        int[] a = new int[10];
        int n = ss.Length;      // количество введенных чисел-подстрок
        int i = 0;
        foreach (string b in ss)  a[i++] = Convert.ToInt32(b);
        Console.WriteLine ("\nМассив чисел:");
        for (int j=0;j<n;j++) Console.WriteLine (a[j]);
     }
  }
}
```



4.2.2 Динамические строки типа StringBuilder

Пространство имен: System.Text.

Примеры объявления строк:

```
StringBuilder s = new StringBuilder();                // 16 байт
StringBuilder s = new StringBuilder("abcde"); // строка
                        // инициализирована строковой константой
StringBuilder s = new StringBuilder(100); // пустая строка,
                                    // отведено место для 100 байт
StringBuilder s = new StringBuilder("ab",100); // строка
      //инициализирована строковой константой, но место для 100 байт
```

В отличие от string динамическая строка реализована как список строк, т.е. при превышении емкости строки добавляется еще кусок и ссылка на него.

Компоненты класса StringBuilder

Компонент	Вид	Описание
Append	Метод	Добавление в конец строк, подстрок, массивов символов и любых встроенных типов
AppendFormat	Метод	Добавление форматированной строки в конец
Capacity	Свойство	Получение или задание емкости буфера. Если значение меньше количества символов в строке или больше максимального, то генерируется исключение ArgumentOutOfRangeException
Insert	Метод	Вставка подстроки в строку с указанного индекса
Length	Свойство	Количество символов в строке
MaxCapacity	Свойство	Максимальный размер буфера = **Int32.MaxValue= =2147483647**
Remove	Метод	Удаление подстроки с заданной позиции
Replace	Метод	Замена всех вхождений подстроки или символа на новые подстроку или символ
ToString	Метод	Преобразование в строку типа string

Пример

Ex4.07

```
using System;
using System.Text;
namespace Ex4_07
{ class Program
    {   static void Main()
        {
            Console.Write( "Введите зарплату: " );
            double salary=double.Parse(Console.ReadLine());
            StringBuilder a = new StringBuilder();
            a.Append("зарплата ");
            a.AppendFormat( "{0, 6:C} – в год {1, 6:C}",
                                        salary, salary * 12 );
            Console.WriteLine( a );
            a.Replace( "руб.", "тыс.$" );
            Console.WriteLine("А лучше было бы: "+a);
        }
    }
}
```

MonoDevelop External Console

```
Введите зарплату: 50000
Зарплата 50 000,00 руб. - В год 600 000,00 руб.
А лучше было бы: Зарплата 50 000,00 тыс.$ - В год 600 000,00 тыс.$

Press any key to continue...
```

4.2.3 Регулярные выражения

Регулярное выражение (regular expression) — шаблон, по которому выполняется поиск и редактирование фрагмента текста.

Примеры:

тег html: **`<\w+>`**

номер телефона: **`[0-9]{3}-[0-9]{2}-[0-9]{2}`**

или **`\d{3}-\d{2}-\d{2}`**

Язык описания регулярных выражений содержит символы двух видов:

- *обычный символ* представляет в выражении сам себя,
- *метасимвол* представляет :
  - класс символов, например: **\d** (цифра), **\w** (буква или цифра);
  - уточняющий символ, например: ^ (в начале), $ (в конце);
  - квантификатор (повторитель) например: + (повторение), {3} (3 раза).

Примеры шаблонов :

- для поиска слова «язык»: **`"язык"`**;
- двух идущих подряд цифр : **`\d\d`**;
- фрагментов вида Вариант 1, … Вариант 9 : **`Вариант \d`**;
- фрагментов вида Вариант 1, Вариант 23, Вариант 719 : **`Вариант \d+`**

Метасимволы. Класс символов

. (точка) - любой символ, кроме \n;

[] - любой *одиночный* символ из последовательности внутри скобок.

[^] - любой *одиночный* символ, не входящий в последовательность внутри скобок.

\w - любой алфавитно-цифровой символ, т.е. символ из множества прописных и строчных букв и десятичных цифр;

\W - любой **не** алфавитно-цифровой символ, то есть символ, не входящий в множество прописных и строчных букв, а также десятичных цифр:

\s - любой пробельный символ, например, пробел, табуляция (\t, \v), перевод строки (\n, \r), новая страница (\f);

\S - любой **не** пробельный символ, то есть символ, не входящий в множество пробельных;

\d - любая десятичная цифра;

\D - любой символ, не являющийся десятичной цифрой.

Метасимволы

Уточняющие метасимволы

^ - следует искать только в начале строки;

$ - следует искать только в конце строки;

\b - начинается или заканчивается на границе слова (т.е. между символами, соответствующими \w и \W);

\B - не должен встречаться на границе слова.

Повторители

* - 0 или более повторений предыдущего элемента;

+ - 1 или более повторений предыдущего элемента;

? - 0 или 1 повторений предыдущего элемента;

{**n**} - ровно **n** повторений предыдущего элемента;

{**n,**} - по крайней мере **n** повторений предыдущего элемента;

{**n,m**} от n до m повторений предыдущего элемента;

Для задания повторителя не для отдельного символа, а для последовательности применяется **группирование** (с помощью круглых скобок).

Примеры

- целое число (возможно, со знаком):

  `[-+]?\d+` например: **0, -123, +35888;**

- вещественное число (может иметь знак и дробную часть, отделенную точкой):

  `[-+]?\d+\.?\d*` например: **-457.45, 56;**

- номер автомобиля:

  `^[А-Я]{1}\d{3}[А-Я]{2}$` например: **А264ЛР**

- ip-адрес (упрощенно):

  (\d{1,3}\.){3}\d{1,3} например: **45.345.648.34**

- адрес e-mail:

  `\w+\@\w+\.\w{2,4}` например: **gsivanova@gmail.com**

Класс Regex (System.Text.RegularExpressions)

Для работы с регулярными выражениями используется класс Regex:

- посредством вызова статических методов класса Regex (параметры метода содержат входную строку и шаблон);
- посредством создания объекта класса Regex и инициализации его регулярным выражением.

Методы класса Regex:

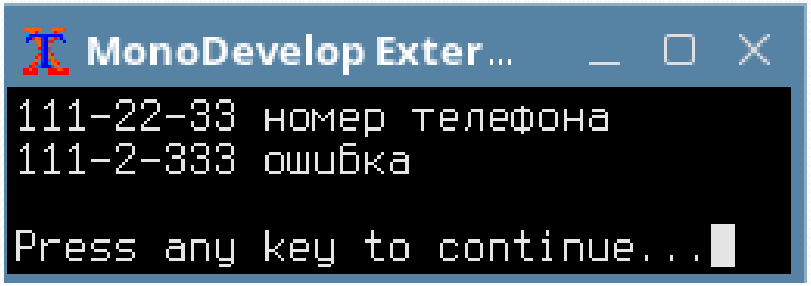
- **`IsMatch()`** - определяет, встречается ли в тексте фрагмент, соответствующий шаблону регулярного выражения (статический метод).
- **`Match()`** и **`Matches()`** - извлекают из текста одно или все вхождения фрагментов, соответствующих шаблону:
  - Match() возвращает объект класса Match с информацией о совпадении,
  - Matches() возвращает коллекцию MatchCollection, в которую входят объекты Match для *всех* совпадений в проанализированном тексте;
- **`Replace()`** - заменяет соответствующий шаблону фрагмент текста на заданный;
- **`Split()`** - разделяет текст на массив строк по заданному разделителю.

Пример использования метода Regex.IsMatch()

Ex4.08

```
using System;
using System.Text.RegularExpressions;
namespace Ex4_08
{
    class Program
    {
        public static void Main()
        {   string[] txt = { "111-22-33", "111-2-333"};
            string reg = @"^\d{3}-\d{2}-\d{2}$";
            foreach (string s in txt)
            {  if (Regex.IsMatch(s,reg))
                    Console.WriteLine("{0} номер телефона", s);
               else Console.WriteLine("{0} ошибка", s);
            }
        }
    }
}
```

Статический метод

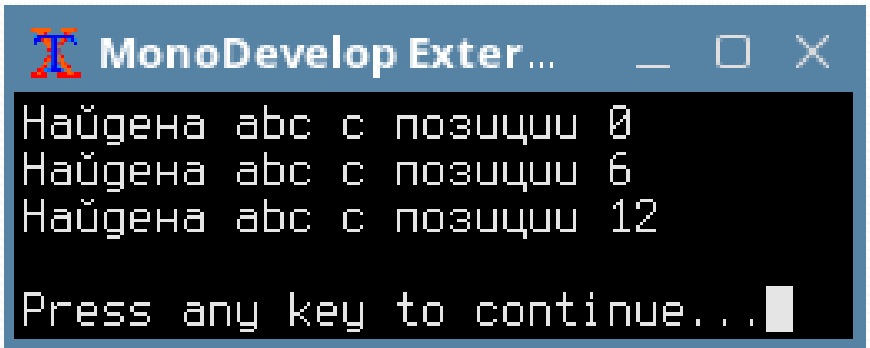


Пример использования метода Regex.Matches()

Ex4.09

```
using System;
using System.Text.RegularExpressions;
namespace Ex4_09
{
  class Program
  {
     public static void Main()
     {
         string txt = "abc123abc456abc789";
         string reg = "abc";   // искомая строка
         foreach (Match m in Regex.Matches(txt, reg))
            Console.WriteLine("Найдена {0} с позиции {1}",
                                                m.Value, m.Index);
     }
  }
}
```

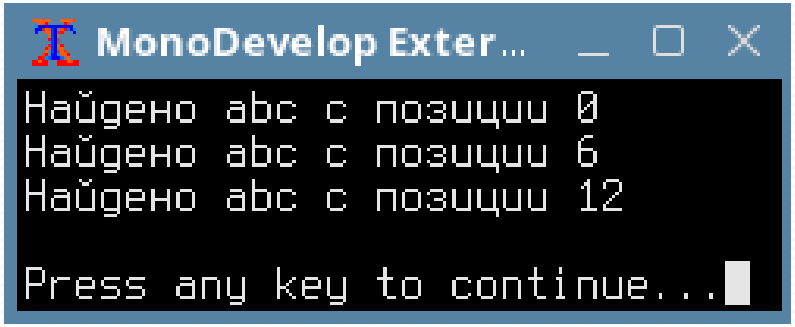
Статический метод



Пример использования класса Regex.Match

Ex4.10

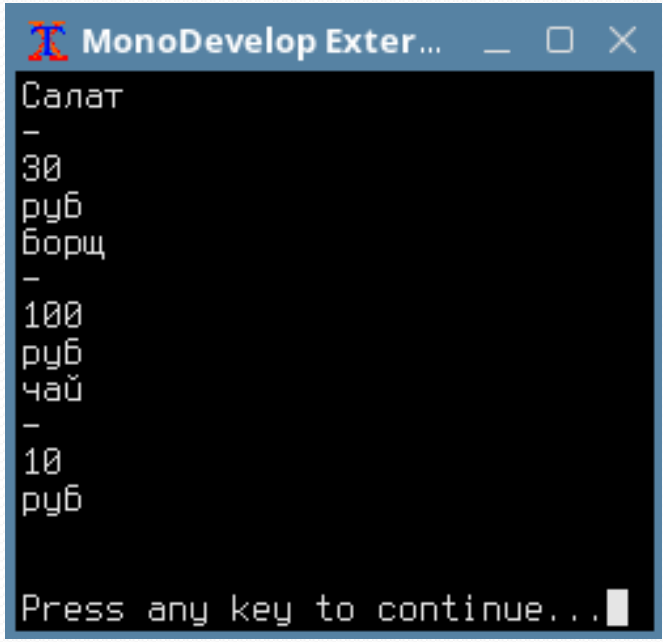
```
public static void Main()
{
   string txt = "abc123abc456abc789";
   string reg = "abc";
        // создание экземпляра класса Match
   Match m = Regex.Match(txt, reg);
   while (m.Success)  // пока есть совпадения
   {
      Console.WriteLine("Найдено {0} с позиции {1}",
                                m.Value, m.Index);
      m = m.NextMatch();
   }
}
```



Пример использования метода Split для разбиения строки на слова

Ex4.11

```
using System;
using System.Text.RegularExpressions;
namespace Ex4_11
{
  class Program
  {
     public static void Main()
     {
        string txt =
         "салат – 30 руб.,борщ – 100 руб.,чай – 10 руб.";
        string reg = "[ -,.]+";
        Regex r = new Regex(reg);
        string[] words = r.Split(txt);
        foreach ( string s in words )
           Console.WriteLine(s);
      }
   }
}
```

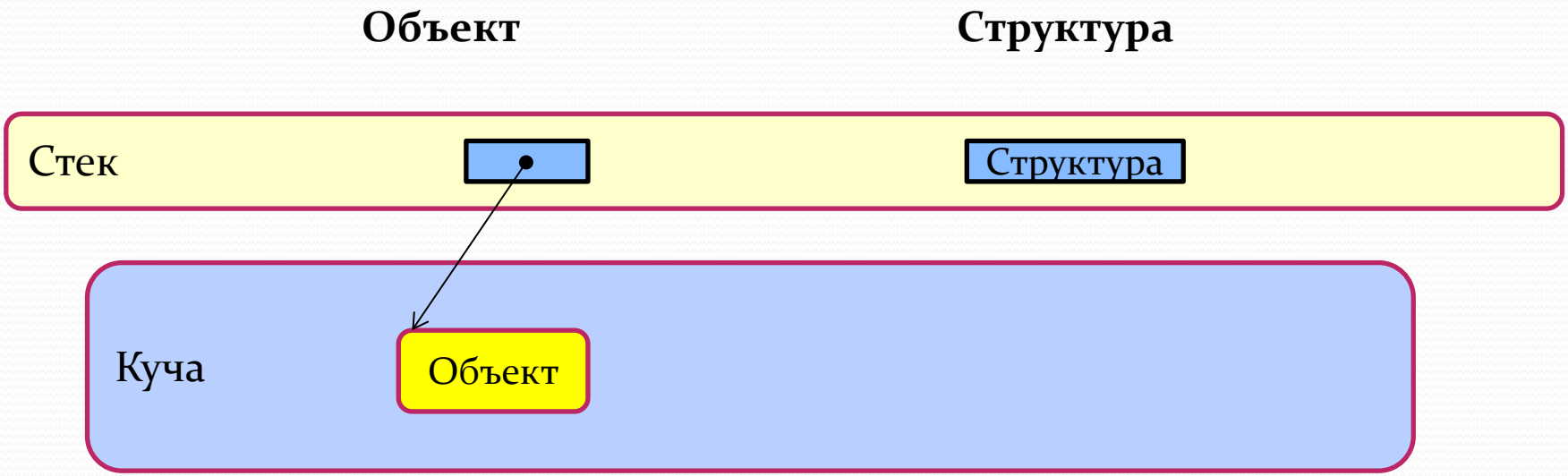


Обычный метод

4.3 Структуры

Структура – эффективный (не требует динамического выделения памяти) тип данных, аналогичный классу, но относящийся к типам-значениям.

Область применения: типы данных с небольшим количеством полей.



Синтаксис описания структуры

Синтаксис:

[**Атрибуты**][**Спецификаторы**] **`struct`** **Имя: Интерфейсы Тело**

Тело структуры может состоять из констант, полей, методов, свойств, событий, индексаторов, операций, конструкторов и вложенных типов.

Спецификатор	Описание
`public`	Доступ не ограничен.
`internal`	Доступ из классов и структур сборки. *Подразумевается по умолчанию.*
`private`	Для вложенных структур. Доступ только из материнских класса или структуры. *Подразумевается по умолчанию для вложенных структур.*
`new`	Для вложенных структур. Задает новое описание структуры взамен унаследованного.

Свойства структур

Особенности:

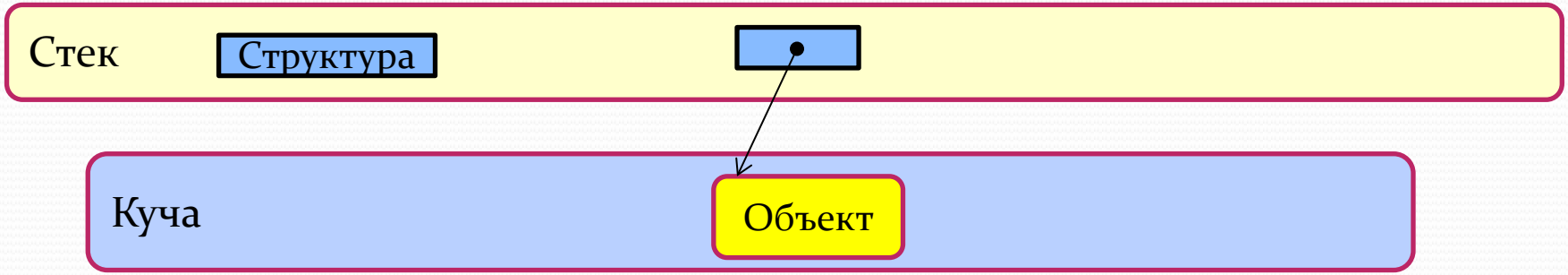
- от структур невозможно наследовать, но структура может реализовывать интерфейсы:
  - не могут быть абстрактными,
  - бесплодны по умолчанию,
  - не могут иметь виртуальных методов;
- в структуре нельзя инициализировать значения и определять конструктор по умолчанию, неявный конструктор обнуляет все поля;
- в структуре нельзя определять деструктор (лишено смысла);
- параметр this для структуры – значение.

Упаковка и распаковка данных типов-значений

При выводе экземпляра структуры на экран выполняется ***упаковка***, то есть неявное преобразование в ссылочный тип. Упаковка применяется и в других случаях, когда структурный тип используется там, где ожидается ссылочный, например, при преобразовании экземпляра структуры к типу реализуемого ею интерфейса. При обратном преобразовании – из ссылочного типа в структурный – выполняется ***распаковка***.

Пример:

```
int valueToBox = 123;
object boxed = valueToBox;          // упакован int
valueOutOfBox = (int)boxed;         // распакован
Console.WriteLine(valueOutOfBox);   // вывод "123"
```



Пример

Ex4.12

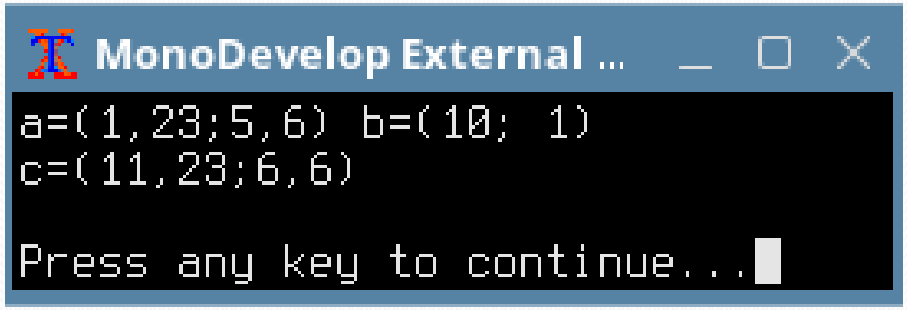
```
using System;
namespace Ex4_12
{
  struct Complex
  {
      public double re, im;
      public Complex( double re_, double im_ )
      {
          re = re_; im = im_;
      }
      public static Complex operator+(Complex a, Complex b)
      {
          return new Complex(a.re + b.re, a.im + b.im );
      }
      public override string ToString()
      {
          return ( string.Format( "({0,2:0.##};{1,2:0.##})",
                                                  re, im ));
      }
  }
```

Пример

```
class Program
{
        static void Main()
        {
                Complex a = new Complex(1.2345, 5.6);
                Complex b,c;
                b.re = 10;
                b.im = 1;
                Console.WriteLine("a="+a+" b="+b);
                c=a+b;
                Console.WriteLine("c="+c);
        }
    }
}
```

Упаковка структуры

Упаковка структуры

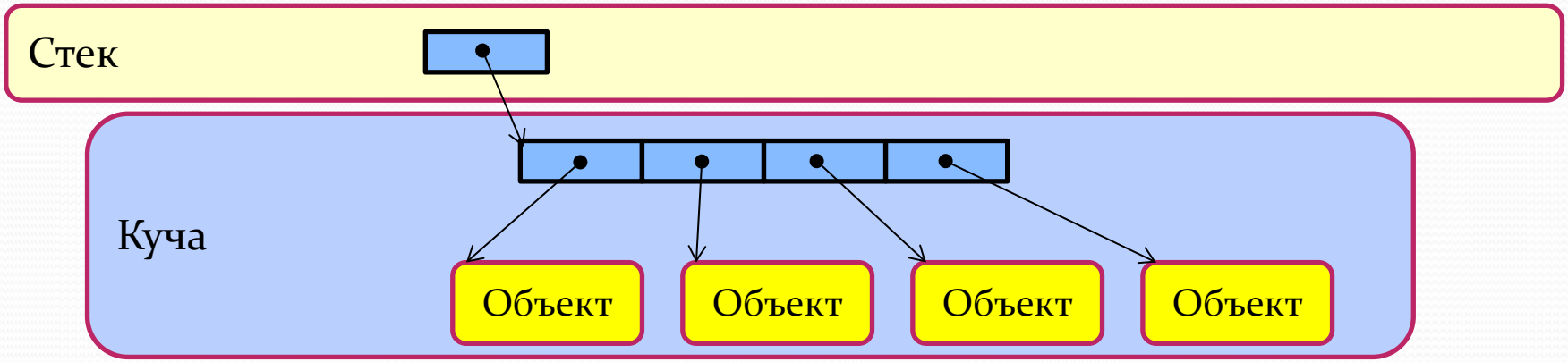


Массив структур

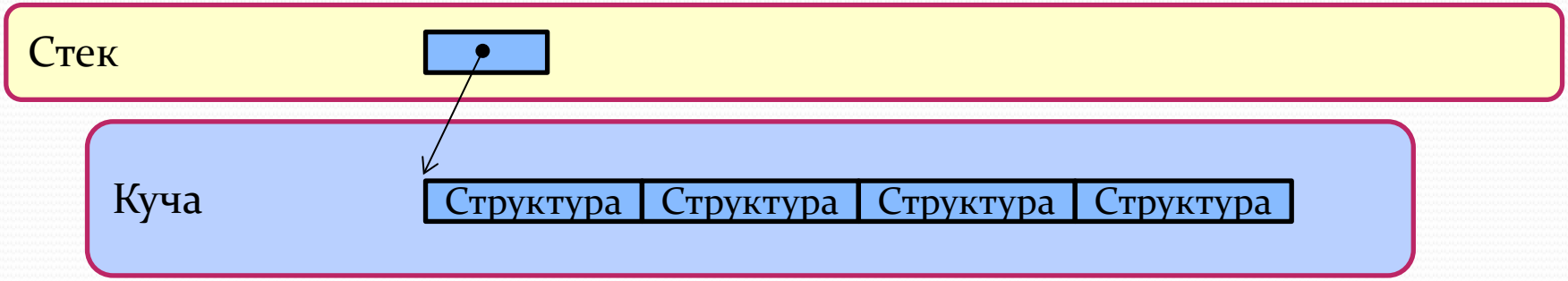
При создании массива объектов память динамически выделяется под массив указателей и под каждый объект.

При создании массива структур – только под сам массив.

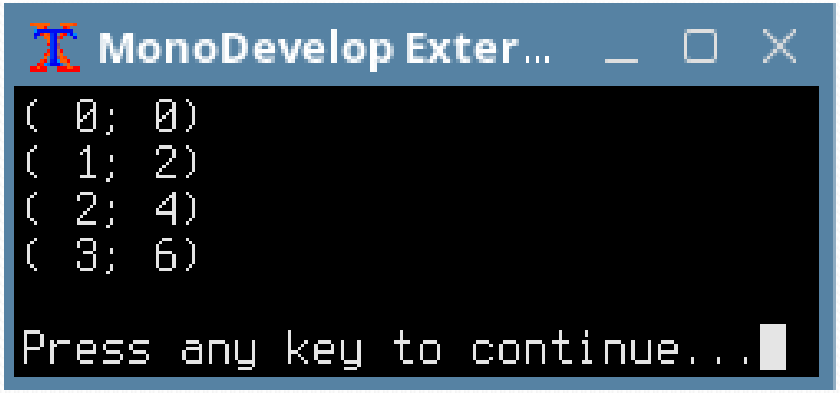
Массив объектов



Массив структур



Пример Массив структур

Ex4.13

```
Complex [] mas = new Complex[4];
for(int i=0;i<4;++i)
{
  mas[i].re = i;
  mas[i].im = 2*i;
}
foreach (Complex elem in mas) Console.WriteLine(elem);
```